Four-Attractor Micro-Prompts for Stable LLM Coding Introducing the FCBP Stability Compass as a Concept Attractor

Introducing the FCBP Stability Compass as a Concept Attractor

Abstract

LLM-assisted coding often fails in repeatable ways: edits ripple across modules, intent becomes hard to read, structure drifts into hacks or over-engineering, and "quick fixes" create future rewrite pressure. This note introduces a tiny set of one-line prompts built from four strong attractor terms—**Flow, Clarity, Balance, Pressure**—and binds them into a single higher-level schema: the **FCBP Stability Compass**. The Compass acts as a *concept attractor hint*: instead of four disconnected slogans, the LLM is nudged to treat them as **four coupled constraints** that jointly define "stable change." The result is a portable, low-friction prompt pack usable by vanilla LLMs without any SMFT/PORE background.

---

1) The Concept Attractor: FCBP Stability Compass

FCBP is a memorable "one-object" frame that packages four attractors into a single mental controller:

- **F — Flow integrity:** minimize ripple; keep interfaces stable; avoid unintended coupling.
- **C — Clarity (low collapse latency):** intent becomes obvious quickly; readable diffs; small functions; explicit names.
- **B — Balance:** neither hacks nor unnecessary abstraction; "justified structure."
- **P — Pressure (low regime pressure):** avoid debt, dependency bloat, brittle constraints that force future rewrites.

Why the Compass matters

Strong words already steer an LLM. But **a named bundle** ("Compass") adds an extra effect: it encourages the model to treat the set as **one cohesive rule**, not four optional vibes. That "binding" is the point: you're not only giving attractor words—you're giving a *coupled framework*.

---

2) The Micro-Prompts (copy-paste)

Concept header (the binder)

Use the FCBP Stability Compass: treat Flow, Clarity, Balance, and Pressure as four *coupled constraints*; optimize them jointly.

Core 3 execution prompts (one line each)

Write code
Write code with FCBP: *flow integrity* (local changes, stable interfaces), *clarity* (clear names, small functions), *structural balance* (simple, justified abstractions), and low *regime pressure* (minimal deps, easy to extend).

Modify code
Modify with FCBP using the smallest change: preserve *flow integrity* (no ripple/API break), increase *clarity* (make intent explicit), keep *balance* (no hacks/over-engineering), and lower *pressure* (avoid new coupling/deps).

Debug code
Debug with FCBP: reproduce → fix root cause → add regression test; preserve *flow integrity*, improve *clarity*, maintain *balance*, and prevent *pressure* (no brittle band-aids).

---

3) Usage: quick, practical patterns

Pattern A — "One-shot stable"

Paste **Concept header + one Core prompt + task details**.

Example skeleton

1. *FCBP Concept header*
2. *Modify code prompt*
3. Task + files + constraints

Pattern B — "Fast loop" (still lightweight)

Use the Compass once, then run a tiny loop:

1. **Plan (3 steps)**
2. **Implement (Write/Modify/Debug)**
3. **Prove (tests / regression)**
4. **Self-review (top 3 risks in F/C/B/P)**

This stays "quick and easy," but stops the most common failure mode: *jumping straight to code without scope + proof*.

---

4) The logic behind the design (brief)

A) Two-layer control in plain English

Even without theory names, coding always has two layers:

- **What you intend** (requirements, invariants, interfaces)

- **What the system becomes after change** (ripples, coupling, maintainability, future pressure)

FCBP explicitly targets the second layer *while* keeping the first layer legible.

B) "Strong attractors" + "binder schema" = two-stage steering

This prompt tech has two levers:

1. **Lexical attractors** (Flow / Clarity / Balance / Pressure)
   → strong, widely understood quality magnets.

2. **A binder concept** (the "FCBP Stability Compass")
   → turns four magnets into **one structured controller**, increasing consistency across outputs and reducing checkbox behavior.

In practice, this often yields a *second-order stabilizing effect*: the model is less likely to over-optimize one dimension (e.g., clarity) while quietly breaking another (e.g., flow via API ripples).

C) Why "Debug" includes a regression test

"Reproduce → fix → regression test" is the smallest portable proof protocol. It converts "I think it's fixed" into "the system will stay fixed," lowering Pressure and preserving Flow.

---

5) Failure modes and the built-in fuse (keep it honest)

When the Compass can "over-stabilize"

If treated as dogma, FCBP may become overly conservative (e.g., avoiding necessary refactors).

Add a one-line anti-dogma fuse (optional)

If FCBP goals conflict, prioritize correctness + test proof first, then explain the tradeoff and choose the smallest irreversible change.

This keeps the Compass powerful without trapping the model in "always-minimal" behavior.

---

6) What you actually "inject" into the LLM

You're not injecting a niche theory name. You're injecting a **portable concept attractor**:

- **A named package** (FCBP Stability Compass)
- **Four coupled constraints** that feel like "engineering invariants"
- **A micro protocol** (especially in debugging) that forces proof

That combination is the practical form of "Perspective-of-Everything style" steering: a small, reusable controller that applies across languages, stacks, and tasks—without requiring the model to know your deeper framework vocabulary.

Appendix

Single-line pack variants (each is one line total)

1. **Write code (FCBP single-line)**
   Write with the FCBP Stability Compass—optimize Flow (local change, stable interfaces), Clarity (obvious intent), Balance (justified structure), and low Pressure (minimal deps, easy extension).

2. **Modify code (FCBP single-line)**
   Modify using the FCBP Stability Compass—make the smallest correct diff that preserves Flow (no ripple/API break), improves Clarity, keeps Balance (no hacks/over-engineering), and lowers Pressure (avoid new coupling/deps).

3. **Debug code (FCBP single-line)**
   Debug using the FCBP Stability Compass—reproduce → fix root cause → add regression test; preserve Flow, increase Clarity, maintain Balance, and prevent Pressure (no brittle band-aids).

4. **Code review (FCBP single-line)**
   Review with the FCBP Stability Compass—check Flow (ripples), Clarity (intent), Balance (abstractions), Pressure (future debt), then list top 3 risks with fixes.

5. **Tests (FCBP single-line)**
   Write tests with the FCBP Stability Compass—lock behavior with minimal, stable coverage (happy path + key edges) while minimizing Pressure (no flaky deps/mocks) and keeping Flow.

6. **Plan-before-code (FCBP single-line)**
   Plan with the FCBP Stability Compass—state assumptions, define the stable interface, then give a 3-step plan that minimizes Flow ripple and future Pressure.

7. **Refactor (behavior-preserving, FCBP single-line)**
   Refactor with the FCBP Stability Compass—preserve behavior while improving Clarity and Balance, reducing coupling for Flow, and lowering long-term Pressure.

8. **Anti-dogma fuse (FCBP single-line, optional)**
   If FCBP goals conflict, prioritize correctness + test proof first, explain the tradeoff, then choose the smallest irreversible change.

Reference

OSF Project : Minimal Regime Coordinates for Dissipative Dynamics
https://osf.io/nq9h4/overview

Paper 1 *A Minimal Intrinsic Triple (ρ, γ, τ) as Control Coordinates for Open Gradient-Flow Systems* Paper 2 *A Minimal Intrinsic Triple (ρ, γ, τ) as Control Coordinates for Open Gradient-Flow Systems*
Paper 3 *The Ξ-Stack Universal Viewpoint _ A Two-Layer (Σ_ Ξ) Operator Playbook in (ρ, γ, τ) with Probe–Couple–Pump–Switch Control and a Falsifiability Harness*
Paper 4 *Four-Force Generator Decomposition for Closed-Loop Systems _ A Bottom-Up Protocol to Compile Ξ=(ρ,γ,τ) and Verify Control via Pump–Probe–Switch–Couple*
Paper 5 *The Post-Ontological Reality Engine (PORE)*
Paper 6 Post-Ontological Observer Engineering _ Compiling Self-Referential Quantum Observers into the PORE Protocol for Collapse, Agreement, and Control

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.2, X's Grok language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.